

RI204

Windows programiranje

dr.sci. Edin Pjanić

- Windows grafika
 - Graphics Device Interface (GDI)-osnovni koncepti
 - Device Context (DC)
 - Funkcije:
 - BeginPaint – EndPaint
 - GetDC - ReleaseDC
 - Bojenje prozora
 - Poruka WM_PAINT
 - Ispis teksta

- GDI je API koji obezbjeđuje rad sa grafičkim uređajima kroz jedinstveni interfejs neovisno od tipa i mogućnosti korištenog uređaja:
 - Video prikaz
 - Printer
 - Ploter
 - ostalo...
- Kroz GDI moguće je crtati
 - Linije i krive
 - Obojene oblasti
 - Bitmape
 - Tekst

- DC je Windows apstrakcija za uređaj na kojem se vrši prikaz grafike. Ima dvojaku namjenu:
 - Da obezbijedi jedinstveni interfejs sa uređajima bez obzira na njihova svojstva
 - Čuva često korištene attribute za crtanje (boje, olovke, četkice itd...) kako iste ne bi morali prosljeđivati prilikom specifičnih poziva za crtanje
- DC je u suštini struktura koja je interna za Windows OS a može se mijenjati samo preko HDC, handle-a na DC strukturu.

HDC za određeni prozor u okviru video okruženja moguće je dobiti na dva načina, pomoću funkcija:

- `BeginPaint` i `GetDC`

Korištenje DC uvijek uključuje par funkcija:

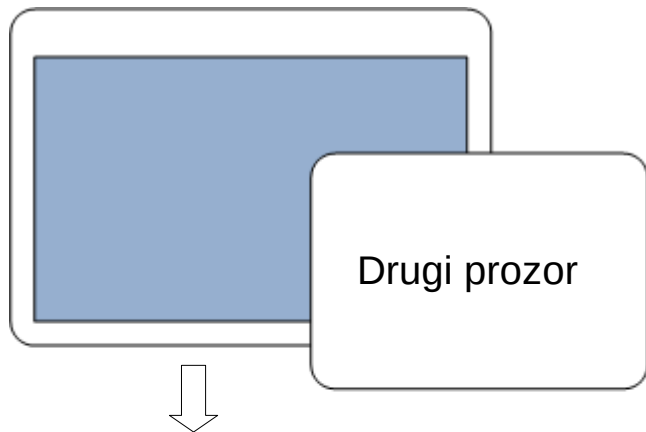
- Prvi par (koristi se pri obradi poruke `WM_PAINT`):
 - `BeginPaint`
 - `EndPaint`
- Drugi par (koristi se u drugim okolnostima):
 - `GetDC`
 - `ReleaseDC`

- Prikaz izlaza u prozoru moguć je kada window procedura primi poruku WM_PAINT, koja indicira da je potrebno bojenje klijent područja prozora.
- WM_PAINT obavezno se šalje:
 - kada se otkriva pokriveni prozor koji je prvobitno bio dijelom ili u potpunosti pokriven,
 - kada se promijene dimenzije prozora,
 - na skrolanje prozora,
 - na eksplicitan poziv funkcije InvalidateRect.
- WM_PAINT može biti poslana kada se otkriva dio prozora usljed micanja dialoga, zatvaranja menija, otvaranja tooltipa.
- WM_PAINT se ne šalje za događaje micanja kursora po površini prozora i prevlačenja ikone preko prozora.

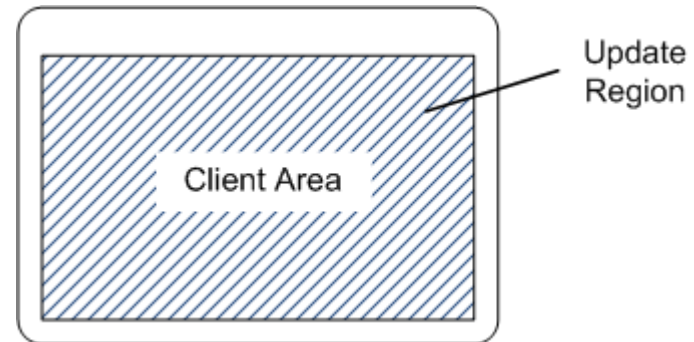
Invalidno područje (invalid region)

7

- Invalidno područje je ono područje prozora koje je potrebno iscrtati (nastaje usljed preklapanja prozora, promjene dimenzija prozora itd.)

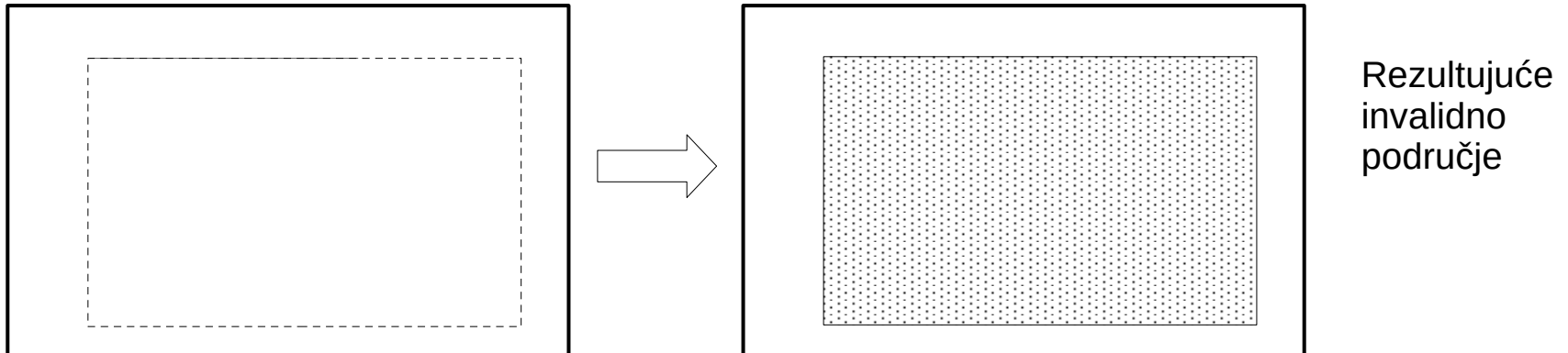


Nakon kreiranja i prikazivanja prozora:



Invalidno područje (invalid region)

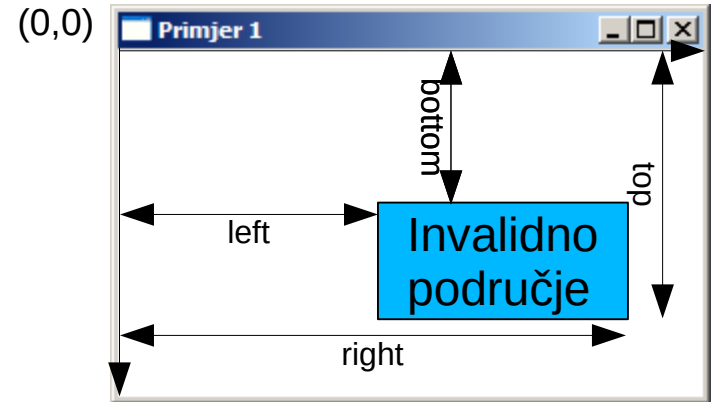
- Kada se detektuje nastanak ovakvog područja, Windows šalje WM_PAINT poruku u red poruka (message queue) aplikacije za procesiranje u window proceduri.
- U redu poruka može biti samo jedna WM_PAINT poruka.
- Ako potencijalno nastane više invalidnih područja tokom procesiranja Windows OS računa jedno područje koje obuhvata sva invalidna područja.
- WM_PAINT je poruka niskog prioriteta i šalje se samo ako u redu poruka aplikacije nema drugih poruka.



```
case WM_PAINT:  
    hdc = BeginPaint (hwnd, &ps) ;  
    //pozivati funkcije za crtanje  
    EndPaint (hwnd, &ps) ;  
    return 0 ;
```

- Gdje su objekti hdc i ps definirani na sljedeći način:
 - PAINTSTRUCT ps;
 - HDC hdc;
- Pozivom funkcije BeginPaint validira se invalidno područje i briše njegova pozadina, te se popunjavaju polja ps strukture a vraća se handle na DC prozora.
- Windowsi vrše crtanje (invalidnog područja) tek po pozivu funkcije EndPaint
- **Obavezno** treba koristiti BeginPaint i EndPaint jer se na taj način validira invalidno područje.

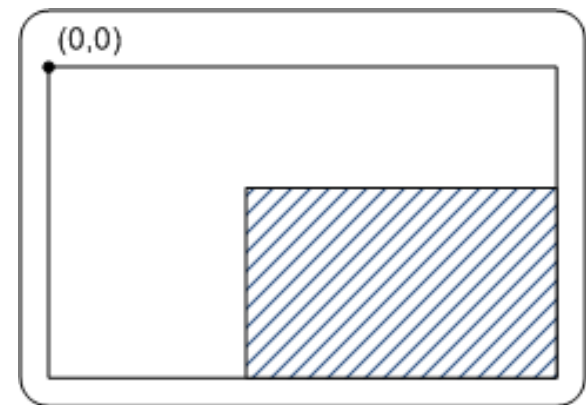
```
typedef struct tagPAINTSTRUCT
{
    HDC          hdc ;
    BOOL         fErase ;
    RECT         rcPaint ;
    BOOL         fRestore ;
    BOOL         fIncUpdate ;
    BYTE         rgbReserved[32] ;
} PAINTSTRUCT ;
```



- Samo prva tri člana su namijenjena za korištenje od strane korisnika:
 - hdc je handle na DC prozora
 - fErase je flag koji indicira da li je brisana pozadina invalidnog područja
 - rcPaint je struktura koja definiše invalidno područje sa poljima: left, right, top i bottom.
 - Nakon poziva BeginPaint, Windows dozvoljava crtanje samo unutar invalidnog područja.

- **Obavezno** treba koristiti `BeginPaint` i `EndPaint` jer se na taj način **validira invalidno područje**.
- U suprotnom, `WM_PAINT` poruka će se ponovo naći u redu poruka.
- Ukoliko naša aplikacija ne obrađuje `WM_PAINT` poruku, potrebno je proslijediti je u `DefWindowProc`, koja će je adekvatno tretirati na sljedeći način:

```
case WM_PAINT:  
    BeginPaint (hwnd, &ps) ;  
    EndPaint (hwnd, &ps) ;  
    return 0 ;
```

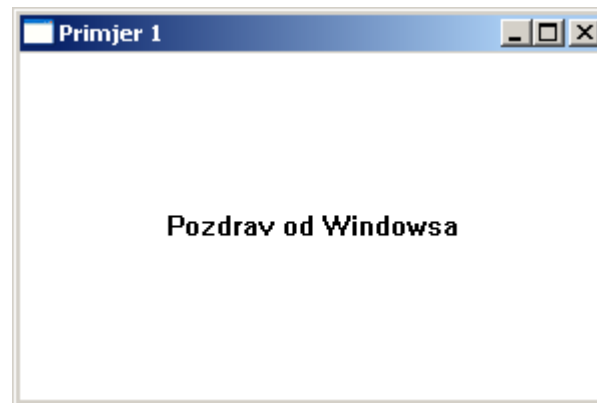


- WM_PAINT poruka je niskog prioriteta, zbog toga je nekada potrebno dugo čekati na osvježavanje ekrana.
- Za situacije kada je potrebno pristupiti DC-u van WM_PAINT poruke (za uzimanje nekih atributa DC-a) ili za brzo crtanje u bilo kojem dijelu klijent područja bez ograničenja na invalidna područja koriste se kombinacija funkcija GetDC i ReleaseDC. Funkcija GetDC ne zahtijeva ps strukturu.
 - Koristi se obično za brzo procesiranje komandi sa tastature

```
hdc = GetDC (hwnd) ;  
// ovdje idu pozivi za GDI funkcije  
ReleaseDC (hwnd, hdc) ;
```

- Za ispis teksta unutar DC-a često se koristi funkcija:
 - `TextOut (hdc, x, y, psText, iLength) ;`
 - `hdc` je handle za DC prozora
 - `x` i `y` su koordinate gornjeg lijevog ugla pravougaonika unutar kojeg se nalazi tekst
 - `psText` adresa C-stringa (niza karaktera) koji se ispisuje
 - `iLength` je broj karaktera u stringu
- Funkcija ne procesira kontrolne znakove unutar stringa.

- Napisati program koji u klijent području ispisuje poruku kao na donjoj slici.
- Poruka treba da ostane na sredini klijent područja bez obzira na promjenu dimenzije prozora.



Šta bismo mogli koristiti?

15

```
BOOL GetTextExtentPoint32A(  
    [in]   HDC      hdc,  
    [in]   LPCSTR   lpString,  
    [in]   int       n,  
    [out]  LPSIZE    psize  
);  
typedef struct tagSIZE {  
    LONG cx;  
    LONG cy;  
} SIZE, *PSIZE, *LPSIZE;
```

Funkcija za računanje dimenzija prostora kojeg bi tekst zauzeo na ekranu na ekranu.

Pomoćna struktura za dimenzije (teksta).

- WM_SIZE poruka koja se šalje prozoru kad mu se promijene dimenzija prozora. Parametri:
 - wParam - tip promjene dimenzija (minimiziran, maksimiziran, ...)
 - lParam - nove dimenzije kodirane na slj. način:
 - donjih 16 bita – nova širina
 - gornjih 16 bita – nova visina
 - koristiti makroe: LOWORD i HIWORD

Window procedura

16

```
LRESULT CALLBACK WndProc (HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    static int  cxClient, cyClient;
    HDC hdc;
    PAINTSTRUCT ps;
    char poruka[] = "Pozdrav od Windowsa";
    static SIZE sz;
    switch (message)
    {
        case WM_CREATE:
            hdc = GetDC(hwnd);
            GetTextExtentPoint32(hdc,poruka,strlen(poruka),&sz);
            ReleaseDC(hwnd,hdc);
            return 0;
        case WM_SIZE:
            cxClient = LOWORD(lParam);
            cyClient = HIWORD(lParam);
            return 0;
        case WM_PAINT:
            hdc = BeginPaint(hwnd,&ps);
            TextOut(hdc,(cxClient - sz.cx)/2,(cyClient - sz.cy)/2,poruka,strlen(poruka));
            EndPaint(hwnd,&ps);
            return 0;
        case WM_DESTROY:
            PostQuitMessage (0) ;
            return 0 ;
    }
    return DefWindowProc (hwnd, message, wParam, lParam) ;
}
```